

字符串问题选讲

清华大学 宋学伟

2023.7.19



知识回顾

Hash

通常我们采用的是多项式 Hash 的方法，对于一个长度为 l 的字符串 s 来说，我们可以这样定义多项式 Hash 函数： $f(s) = \sum_{i=1}^l s[i] \times b^{l-i} \pmod{M}$ ，其中 b 是一个任选的数， M 是选定的大质数。

可以类比为 b 进制数来理解：abcd $\rightarrow$ (97)(98)(99)(100)

查询字串 Hash 值：

预处理出每个前缀 $s[1...i]$ 的 Hash 值，记作 h_i ，则字串 $s[l...r]$ 的 Hash 值为 $h_r - h_{l-1} \times b^{r-l+1} \pmod{M}$ ，类似于 $34 = 1234 - 12 \times 100$ 。

为防止 Hash 冲突，可以将计算出的 Hash 值对两个质数取模，两个 Hash 值都相等时才认为字符串相同。

最长公共子串

给定 m 个总长不超过 n 的非空字符串，求出所有字符串的最长公共子串长度。

其中 $m, n \leq 10^6$

如果存在长度为 k 的公共子串，那么长度为 $k-1$ 的公共子串也必定存在。因此可以二分最长公共子串的长度，假设当前二分长度为 mid ，将 m 个字符串长度为 mid 的所有子串分别 Hash 得到 m 个 Hash 值的集合，然后求出交集即可。

时间复杂度 $O(n \log n)$

KMP

如果字符串 s 的长度为 d 的前缀 ($1 \leq d < |s|$) 和长度为 d 的后缀相等, 称这个前缀是 s 的一个 **border**, 将前缀 $s[1...i]$ 的最长 border 长度记作 $next(i)$ 。

border 具有以下性质: 字符串 s 的所有 border 长度为 $next(i)$, $next(next(i))$, $next(next(next(i))) \dots$, 基于此可以在 $O(|s|)$ 的复杂度内求解出所有的 $next(i)$ 。

给定字符串 s 和 t , KMP 算法可以在 $O(|s|+|t|)$ 的时间复杂度内查找 t 在 s 中的所有出现位置。更确切地说, 我们可以求得 s 每个前缀 $s[1...i]$ 的后缀与 **t 的前缀** 的最长匹配长度, 事实上求解 $next$ 数组的过程即是求 s 每个前缀 $s[1...i]$ 的后缀与 **s 的前缀** 的最长匹配长度。

USACO15FEB Censoring S

给定字符串 s 和 t ，每次删去 s 中第一次出现的子串 t ，并将剩余部分拼接，重复此过程直到 s 的剩余部分没有 t 的出现。求剩余的字符串。

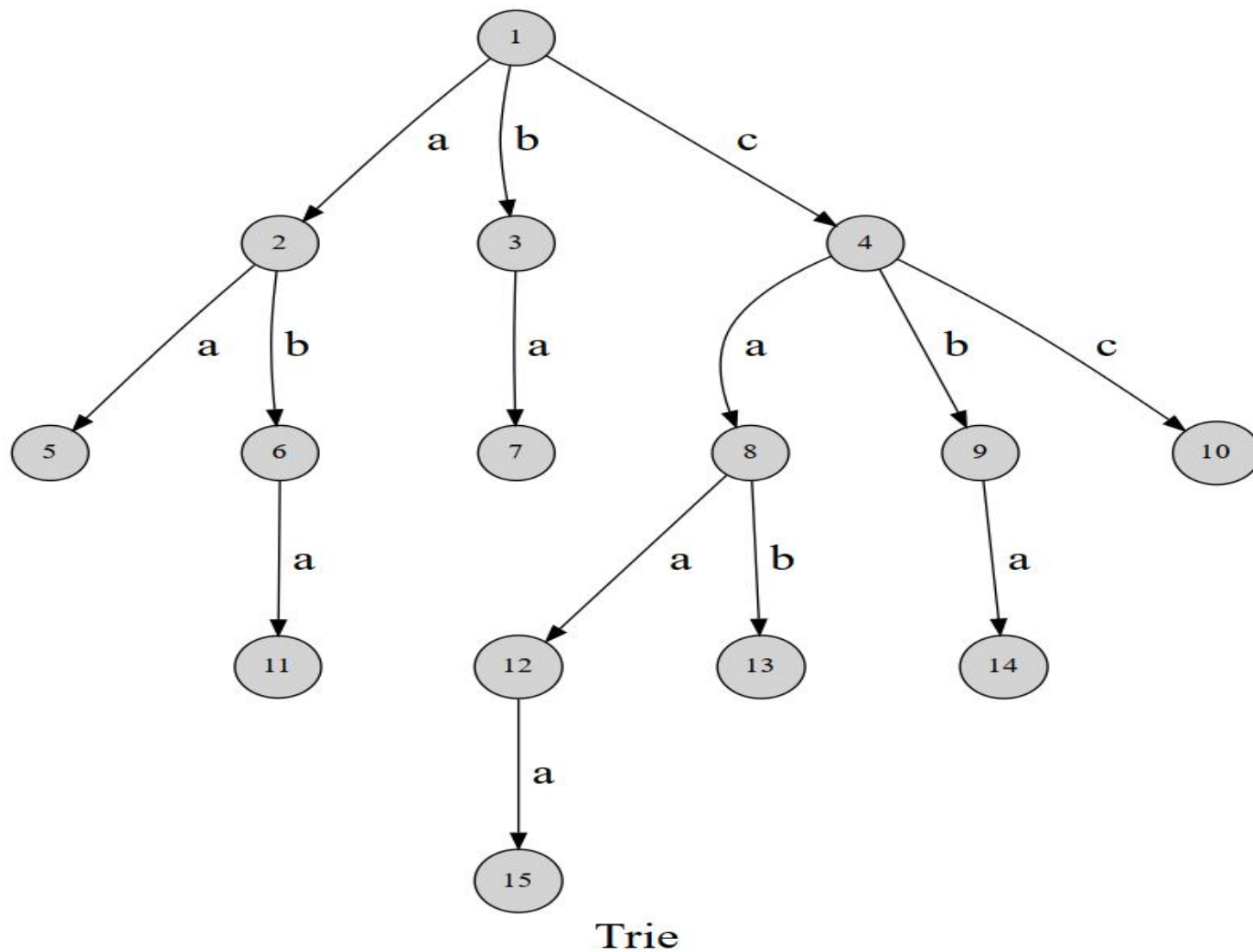
其中 $1 \leq |t| \leq |s| \leq 10^6$

使用 KMP 算法可以求得 s 每个前缀与 t 的最长匹配长度，换言之，我们维护了在 s 的每个位置的**匹配进度**。对于本题，我们在 KMP 过程中将 s 中涉及到的下标依次入栈，并记录每个位置的匹配进度，当匹配上 t 时，将栈顶的 $|t|$ 个下标出栈，然后从栈顶的进度继续匹配即可。

复杂度分析：字符串 s 中每个位置只会遍历一次，而对于匹配进度每次 s 向前一个字符时至多+1， s 中删去 $|t|$ 个字符后至多+ $|t|$ ，每次跳转 next 数组至少 -1，故总复杂度为 $O(|s|)$ 的。

字典树 (Trie)

Trie 上保存有插入的所有字符串的前缀的信息



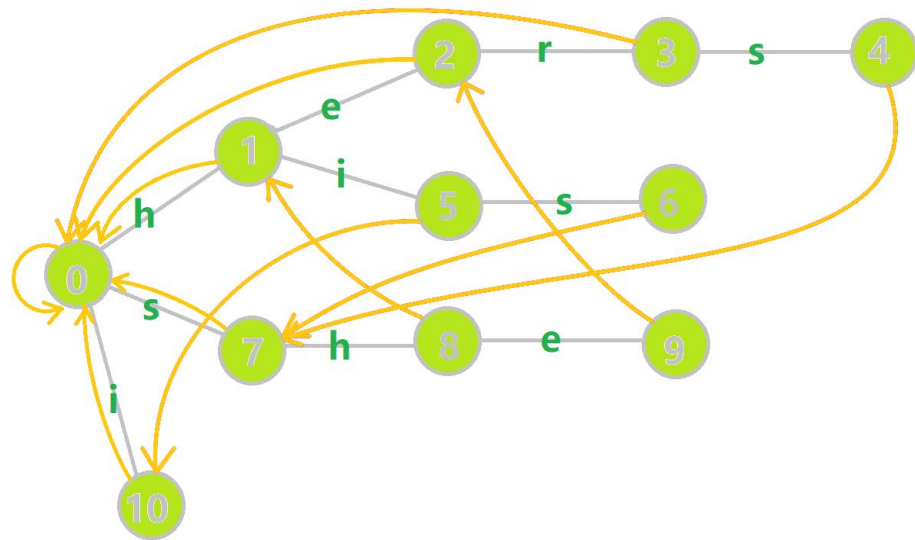
AC自动机——fail树

AC自动机是以 Trie 的结构为基础，结合 KMP 的思想建立的自动机。

AC自动机的两大基本结构：fail 树和 Trie 图

给定若干字符串 $s_1 \dots s_n$ ，将其建成 Trie 后，一个点 u 的 **fail 指针**指向 Trie 上深度更小的一点 v ，满足 v 对应的串是 u 对应串的最长后缀。将所有 fail 指针看作边的话则形成了 fail 树结构。

fail 树的性质：一个点的所有祖先是其后缀，且作为其他某个串的前缀出现。



AC自动机——Trie图

Trie 图是反映一个字符串在AC自动机上匹配过程的一个有向图。

用 $ch(u, c)$ 表示原 Trie 树上的边，Trie图上的边是如下定义的：

$$tr(u, c) = \begin{cases} ch(u, c) & ch(u, c) \neq \emptyset \\ tr(fail(u), c) & ch(u, c) = \emptyset \end{cases}$$

用一个字符串 t 在 Trie 图上走，最终到达的节点为 t 在 Trie 树上能够匹配到的最长后缀，这个后缀是串 $s_1 \dots s_n$ 中某个串的前缀。

JSOI2012 玄武密码

给定字符串 s 和 $t_1 \dots t_m$, 对于每个 t_i 求出最长的前缀使得其是 s 的子串。

数据范围: $|s| \leq 10^7$, $m \leq 10^5$, $|t_i| \leq 100$, 字符集 $\{E, S, W, N\}$

s 的子串 —— s 所有前缀的后缀

将 m 个字符串 t 建成AC自动机, 用 s 在其 Trie 图上匹配, 将途径到的每个点在 fail 树上的祖先全部标记, 然后对于每个 t 找到最长的被标记前缀即可。

时间复杂度 $O(|s| + \sum |t_i|)$



题目选讲

NOI2016 优秀的拆分(95%)

给定长度为 n 的字符串 S ，求 S 的所有子串拆成 $AABB$ 形式的方案总和，其中 A, B 都是非空字符串。T 组数据。

$T \leq 10$ ，对于 95% 的数据 $n \leq 2000$ ，字符集 26

记 $f(x)$ 表示以 x 为结尾的 AA 形式的串数量， $g(x)$ 表示以 x 为开头的 AA 形式字符串数量，答案表示为 $\sum_{x=2}^{n-2} f(x)g(x+1)$ 。 $f(x), g(x)$ 通过 Hash 暴力即可 $O(n^2)$ 求出。这样就获得了 95pts。

对于最后 5% 的数据 $n \leq 30000$ ，如何去做呢？

[IOI2008] Type Printer

你需要利用一台可移动的打印机打印出 N 个单词。这种可移动式打印机是一种老式打印机，它需要你将一些小的金属块（每个包含一个字母）放到打印机上以组成单词。然后将这些小金属块压在一张纸上以打印出这个词。这种打印机允许你进行下列操作：

- 在打印机当前词的末端（尾部）添加一个字母；
- 在打印机当前词的尾部删去一个字母（将打印机当前词的最后一个字母删去）。仅当打印机当前至少有一个字母时才允许进行该操作；
- 将打印机上的当前词打印出来。初始时打印机为空，或者说它不含任何带字母的金属块。打印结束时，允许有部分字母留在打印机内。同时也允许你按照任意的次序打印单词。

由于每一个操作都需要一定时间，所以需要你尽可能减少所需操作的总数目（将操作的总数最小化）。

你需要编写一个程序，给定所要打印的 N 个单词，找出以任意次序打印所有单词所需操作的最小数目，并输出一种这样的操作序列。

$N \leq 25000$ ，串长 ≤ 20 ，字符集 26

[IOI2008] Type Printer



将所有字符串建成 Trie，dfs 整棵树即可，由于最后可以保留一个串在打印机内，需要最后遍历最长的一个字符串。

为什么是 Trie？在末端插入（向下 dfs）和末端删除（回溯）形成了很自然的 dfs 树结构，而用树表示所有字符串，最精简的结构就是 Trie。

NOI Online 2021 提高组 积木小赛

给定两个长度为 n 的字符串 s 和 t ，求 t 中有多少个**本质不同**的子串是 s 的子序列。

$n \leq 3000$ ，字符集 26

t 的子串只有 $O(n^2)$ 个，考虑直接从 t 的每个位置 l 向后枚举子串 $t[l \dots r]$ ，如何判定 s 中是否有对应子序列？维护一个位置 pos 表示 $t[l \dots r]$ 作为 s 子序列右端点最小的位置，每次 t 向后添加一个字符 c ，在 s 中找到 pos 后第一个 c 即可（子序列自动机）。

将可以匹配到的 t 的子串的 Hash 值保存，去重即得答案。

时间复杂度 $O(n^2)$ ，子序列自动机部分可以在 $O(26n)$ 的复杂度预处理得到。

NOIP2020 字符串匹配

给定字符串 S ，问有多少不同的非空字符串 A, B, C 满足 $S=ABAB\dots ABC$ 且 $f(A) \leq f(C)$ ，其中 AB 表示 A, B 两个字符串顺序拼接， $f(S)$ 表示 S 中出现奇数次的字符数量。 T 组数据。

$T \leq 5, |S| \leq 10^6$ ，字符集 26

先枚举 AB 的长度 len ，再枚举 AB 重复次数 k ，可以用 Hash 判断 AB 至多重复几次。后缀 $s[klen + 1 \dots |S|]$ 作为 C ，需要查询 $1 \leq i < len$ 中有多少满足 $f(s[1 \dots i]) \leq f(C)$ 。由于 $f \leq 26$ ，可以在 len 增加时 $O(26)$ 的维护 $\leq x$ 的 $f(s[1 \dots i])$ 共有多少个，每次查询则是 $O(1)$ 回答。

由于 $k \leq \left\lfloor \frac{|S|}{len} \right\rfloor$ ，总的枚举量是 $O(|S| \log |S|)$ 的，总复杂度 $O(|S|(\log |S| + 26))$ 。

POI2000 病毒

给定 n 个01字符串 $t_1 \dots t_n$, 判断是否存在一个无限长的01串使得 $t_1 \dots t_n$ 都不是其子串

$n \leq 2000$, $\sum |t_i| \leq 30000$

将 $t_1 \dots t_n$ 建成AC自动机, 用一个无限长的串在其 Trie 图上匹配的过程中显然不能经过这 n 个串所代表的节点, 其次也不能经过这些点在 fail 树上子树中的节点, 我们将这些点删去, 之后如果 Trie 图中还存在一个环, 则无限长的串存在。

在 Trie 图上 dfs, 记录哪些点在当前 dfs 的递归栈中, 如果访问到了一个栈中节点则判定成环 (类似与强连通分量的求解)

NOI2014 动物园

给定长度为 L 的字符串 S ，对于子串 $S[1\dots i]$ 求出既是其前缀又是其后缀，且前缀后缀不重叠的串数量，记作 $num[i]$ 。输出所有 $(num[i] + 1)$ 的乘积，对 $10^9 + 7$ 取模
 $L \leq 10^6$ ，字符集 26，至多 5 组数据

对 S 做一次 KMP 求出 $next$ 数组，然后考虑先对于每个前缀求出不重叠的 border 最长长度 $len[i]$ ，若已知 $len[i - 1]$ ，在向后添加一个字符 c 时，类似 KMP 多次跳 $next$ 直至 border 长度为 0 或可以添加一个 c ，添加 c 后再多次跳 $next$ 直至 $len[i] \leq \frac{1}{2} i$ 即可。

求出 $len[i]$ 后 $num[i] = cnt[len[i]]$ ，其中 $cnt[i] = cnt[next[i]] + 1$ ($next[i] > 0$)
即为长度为 i 的前缀可以跳 $next$ 的次数。总复杂度显然是线性的。

POI2005 SZA-Template

你打算在纸上印一串字母。

为了完成这项工作，你决定刻一个印章。印章每使用一次，就会将印章上的**所有**字母印到纸上。

同一个位置的相同字符可以印多次。例如：用 `aba` 这个印章可以完成印制 `ababa` 的工作（中间的 `a` 被印了两次）。但是，因为印上去的东西不能被抹掉，在同一位置上印不同字符是不允许的。例如：用 `aba` 这个印章不可以完成印制 `abcba` 的工作。

因为刻印章是一个不太容易的工作，你希望印章的字符串长度尽可能小。

输入一个长度不超过 5×10^5 的非空字符串（只包含小写字母），代表要在纸上印的字符。

POI2005 SZA-Template——解法1

记给定的字符串为 s ，所用的印章串显然应该是 s 的一个 border，最直接的思路是，枚举 s 的 border t ，然后将所有 t 出现的位置都印一下，最后不留下未印的空隙即可。

直接枚举的话复杂度过高，考虑从小到大枚举 border t ，出现的位置会不断减少， t 满足条件需要出现位置相邻距离不超过 $|t|$ （都取末尾来计算）。**注意到 t 与子串 $s[p - |t| + 1 \dots p]$ 匹配的充要条件是 t 是前缀 $s[1 \dots p]$ 的 border。**我们将 s 的 next 数组关系建成树（ $0 \dots |s|$ 共 $|s|+1$ 个点， u 与 $\text{next}(u)$ 连边），则一个前缀的所有 border 是其树上除根以外的祖先。

从树根 0 开始沿着下标 $|s|$ 所在的链往下走，每走一步都从出现位置的集合中删去其他子树，可以用链表来维护，时间复杂度是 $O(|s|)$ 的。

POI2005 SZA-Template——解法2

考虑用动态规划的思路来解决，设 $f(i)$ 表示前缀 $s[1...i]$ 的答案，所求即 $f(|s|)$ 。对于 s 来说可能的答案都是 s 的 border，则覆盖 s 就一定能恰好覆盖 $s[1...next(s)]$ 这个前缀，对 $s[1...next(s)]$ 也是同理的，所以 $f(i)$ 取值只能是 i 或者 $f(next(i))$ 。

$f(i) = f(next(i))$ 什么时候成立呢？ s 的最后 $next(i)$ 个字符显然覆盖了，所以需要存在一个 $p \geq i - next(i)$ 使得 $f(p) = f(next(i))$ ，用一个桶来维护每种 f 取值的最大下标即可，时间复杂度同样是线性的。

JSOI2007 文本生成器

给定字符串 $s_1 \dots s_n$ ，求有多少长度为 m 的大写英文字母串 t 至少包含一个 s_i ，答案对 $10^4 + 7$ 取模。

$n \leq 60, m \leq 100, |s_i| \leq 100$ ，字符集 26

考虑求出不包含任意 s 的字符串 t 数量，对所有 s 建立AC自动机，则 t 在 Trie 图上匹配不能经过 $s_1 \dots s_n$ 代表的点及其 fail 树上子树中的点。设 $f(i, u)$ 表示长度为 i 的字符串 t 在AC自动机上最后匹配到 u 的方案数，初始 $f(0, rt) = 1$ ，转移为 $f(i + 1, tr(u, c)) += f(i, u)$ ，其中 $tr(u, c)$ 应当是合法点。最后将所有 u 是合法点的 $f(m, u)$ 相加即得。

时间复杂度 $O(26nm|s|)$

JSOI2007 文本生成器(改)

给定字符串 $s_1 \dots s_n$ ，求有多少长度为 m 的大写英文字母串 t 至少包含一个 s_i ，答案对 $10^4 + 7$ 取模。

$n \leq 20$, $m \leq 10^9$, $|s_i| \leq 10$, 字符集 26

动态规划思路是同理的，但是直接转移不能通过

考察状态转移方程 $f(i + 1, tr(u, c)) += f(i, u) \ (c \in \{A, \dots, Z\})$

可以写出从 $f(i, *) \rightarrow f(i + 1, *)$ 的转移矩阵，大小为 $\sum |s_i| \times \sum |s_i|$ ，矩阵快速幂优化即可

时间复杂度 $O((\sum |s_i|)^3 \log m)$

CF585F Digits of Number Pi

给定长度为 n 的数字串 s 和长度为 d 的不含前导零的数字串 x, y 。

求存在长度至少为 $\lfloor \frac{d}{2} \rfloor$ 的子串是 s 的子串的数字串 $x \leq t \leq y$ 的数量。

$n \leq 1000, d \leq 50$, 答案对 $10^9 + 7$ 取模

注意到 t 的条件等价于 t 中存在一个长度恰为 $\lfloor \frac{d}{2} \rfloor$ 的 s 的子串。 因此可以将 s 中所有

长度为 $\lfloor \frac{d}{2} \rfloor$ 的子串拿出来建立AC自动机，并用 t 来匹配。

对于 t 的上下界限制，先差分变为两个 $\leq r$ 的问题，设 $f(i, u, 0 \text{ or } 1)$ 表示确定了 t 的高 i 位，当前在AC自动机上匹配到 u ，是否有上界限制的方案数，记 r 的高 $i+1$ 位为 e ，状态转移为：

$$f(i, u, 0) \rightarrow f(i + 1, tr(u, c), 0)$$

$$f(i, u, 1) \rightarrow f(i + 1, tr(u, c), 1) \quad (c = e)$$

$$f(i, u, 1) \rightarrow f(i + 1, tr(u, c), 0) \quad (c < e)$$

时间复杂度 $O(10nd^2)$

NOI2011 阿狸的打字机

阿狸喜欢收藏各种稀奇古怪的东西，最近他淘到一台老式的打字机。打字机上只有 28 个按键，分别印有 26 个小写英文字母和 **B**、**P** 两个字母。经阿狸研究发现，这个打字机是这样工作的：

- 输入小写字母，打字机的一个凹槽中会加入这个字母(这个字母加在凹槽的最后)。
- 按一下印有 **B** 的按键，打字机凹槽中最后一个字母会消失。
- 按一下印有 **P** 的按键，打字机会在纸上打印出凹槽中现有的所有字母并换行，但凹槽中的字母不会消失。

例如，阿狸输入 `aPaPBbP`，纸上被打印的字符如下：

```
a
aa
ab
```

我们把纸上打印出来的字符串从 1 开始顺序编号，一直到 n 。打字机有一个非常有趣的功能，在打字机中暗藏一个带数字的小键盘，在小键盘上输入两个数 (x, y) (其中 $1 \leq x, y \leq n$)，打字机会显示第 x 个打印的字符串在第 y 个打印的字符串中出现了多少次。

输入的操作串长 $\leq 10^5$ ， $n \leq 10^5$ ，小键盘询问数 $m \leq 10^5$

NOI2011 阿狸的打字机

显然不能对每次询问做 KMP。按照操作进行模拟，可以建立一棵节点数 $N \leq 10^5$ 的 Trie 树，然后建立 AC 自动机。对于一个询问 (x, y) ，即考虑字符串 y 的所有前缀，看这些点在 fail 树上有多少位于 x 的子树内，通过 dfn 序可以转化为区间询问。

我们考虑对于同一个 y ，回答所有 $(*, y)$ 的询问，但是所有 y 的总长可能很大。考虑到所有 y 构成 Trie，对 Trie dfs，向下递归时将该点的 dfn 位置加 1，回溯时减 1，到达某串 y 处回答相关的所有询问，即在 dfn 序上做若干次区间查询，这个过程可以用树状数组维护。

时间复杂度 $O(26N + (N + m)\log N)$

CF547E Mike and Friends

给定 $s_1 \dots s_n$, q 次询问, 每次给出 k, l, r , 询问 s_k 在 $s_l \dots s_r$ 中一共出现了多少次

$n \leq 2 \times 10^5$, $\sum |s_i| \leq 2 \times 10^5$, $q \leq 5 \times 10^5$

类似地, 对 s 建立AC自动机, 朴素的回答询问的方式即将 $s_l \dots s_r$ 所有前缀处+1, 然后查询 s_k 子树和。为优化, 先将询问拆成两个 $(k, l-1)$ 和 (k, r) , 从 1-n 不断加入字符串, 加入 s_i 时将所有前缀 dfn 处+1, 然后回答所有询问 $(*, i)$, 这个过程同样用树状数组维护即可。

时间复杂度 $O((\sum |s_i| + q) \log(\sum |s_i|))$

CF587F Duff is Mad



给定 $s_1 \dots s_n$, q 次询问, 每次给出 k, l, r , 询问 $s_l \dots s_r$ 在 s_k 中一共出现了多少次

$n \leq 10^5$, $\sum |s_i| \leq 10^5$, $q \leq 10^5$

CF587F Duff is Mad

将上题做法反过来 (× 同样先考虑朴素计算方法 (1) , 将 s_k 前缀处+1, 然后对 $s_l...s_r$ 每一个串进行子树求和, 但是这样即使采用树状数组, 对于每一个 k , 都需要从 1-n 扫描一遍, 复杂度过高。而还有另一种朴素算法 (2) , 将 $s_l...s_r$ 子树内+1, 然后对于 s_k 前缀的每个位置单点求和, 但是直接做也不行。

通过分治可以结合两种做法优化复杂度。记 $m = \sum |s_i|$, 设一个阈值 B , 对于 $s_k > B$ 这样的串至多只有 $\frac{m}{B}$ 个, 对于每一个串采用 (1) , 前缀处+1后直接对于整个树 $O(m)$ 地统计子树和, 然后 1-n 扫一遍即可回答所有与 k 相关的询问, 复杂度 $O(\frac{m^2}{B})$; 对于 $s_k \leq B$, 采用做法 (2) , 从 1-n 扫描, 每次可以 $O(|s_k| \log m)$ 地回答一个询问, 复杂度 $O(qB \log m)$ 。

复杂度平衡, 由于 m, q 同级, 直接取 $B = \sqrt{\frac{m}{\log m}}$, 总复杂度 $O(m\sqrt{m \log m})$

USACO12DEC] First! G

给定 n 个互不相同的小写字母串 $s_1 \dots s_n$ ，现在可以指定 a-z 的大小关系，求哪些串可能成为字典序最小的串

$$n \leq 30000, m = \sum |s_i| \leq 3 \times 10^5$$

将 n 个字符串插入 Trie 树中，按照给定的 a-z 顺序遍历 Trie 树中每个点的儿子，则最先访问到的字符串字典序最小。那么我们直接 dfs Trie 树，每次从某点 u 向下走一个字符 c ，记录 c 应当比 u 的其他出边小，如果用 $c \rightarrow d$ 的有向边来表示 $c < d$ 的关系，则到达某串时若图无环，则可以字典序最小。特别的，如果当前串的前缀存在的话该串一定不能最小。

时间复杂度 $O(26m + 26^2n)$

 **THANKS** 